REPUBLIQUE DU BURUNDI

UNIVERSITE ESPOIR D'AFRIQUE

MODULE MGGT1103

Rapport Technique Détaillé de Fin de Cours

THEMATIQUE 3 :

Conception, Automatisation et Observabilité d'une Plateforme Cloud-Native Distribuée

Fait à Bujumbura 28/04/2026,

Par :

1. MARIAM COBWA Marybella **26200308**
2. MUGISHA WILSON **26200159**
3. MUGISHO BINTU Pascal **26200160**
4. MUNIHIRE NYANGUBA Fidèle **26200232**

Présenté à la Faculté d'Ingénierie et Technologie

Département de Génie et Gestion de Télécommunication

Encadré par :

- Dr. NIBITANGA Romeo

ANNEE ACADEMIQUE 2026

1. Introduction et Cahier des Charges

Ce rapport formalise l'intégralité du travail réalisé dans le cadre du module MGGT1103 : Administration Système, Cloud-Native & DevOps, sous la direction du Dr. NIBITANGA Romeo à l'Université Espoir d'Afrique. Le défi consistait à concevoir, déployer et exploiter une plateforme cloud-native hautement résiliente en nous affranchissant des infrastructures centralisées traditionnelles. L'originalité technique majeure repose sur la mise en place d'un cluster physique distribué interconnectant les ordinateurs portables des membres de l'équipe via un réseau local (LAN) partagé.

Piliers Fondamentaux de la Réalisation :

- **100% Open-Source & Zéro Licence :** Utilisation exclusive d'outils open-source (VirtualBox, Vagrant, Ansible, Docker, K3s, Prometheus, Loki, Grafana, Traefik, Keycloak).
- **Automatisation Intégrale (IaC & GitOps) :** Provisionnement par Vagrant, durcissement et configuration par Ansible (avec chiffrement Ansible Vault), et automatisation des builds par GitHub Actions.
- **Observabilité Proactive & SRE :** Implémentation d'une stack PLG complète avec gestion avancée des règles d'alerte et centralisation des logs en temps réel.

2. Organisation de l'Équipe et Méthodologie Agile

En accord avec la philosophie DevOps, l'équipe s'est auto-organisée en répartissant les responsabilités techniques tout en garantissant une maîtrise transversale par l'ensemble des membres :

- **Pôle Infrastructure & SecOps :** Conception des scripts Vagrant, rédaction des playbooks Ansible de durcissement (hardening) et sécurisation des secrets via Ansible Vault.
- **Pôle Cloud-Native & CI/CD :** Rédaction des Dockerfiles de personnalisation des applications Open-Source et mise en place du pipeline GitHub Actions intégrant les analyses Hadolint et Trivy.
- **Pôle Orchestration & Identité :** Élaboration des manifests Kubernetes, configuration de la passerelle Ingress Traefik et mise en place du SSO Keycloak (OIDC).
- **Pôle Observabilité (SRE) :** Déploiement et paramétrage de la stack PLG (Prometheus, Loki, Grafana) et des règles d'alerte.

3. Topologie Réseau et Architecture Matérielle Distribuée

Pour simuler un environnement de production hautement disponible, le cluster est réparti sur trois machines virtuelles distinctes fonctionnant en mode ponté (Bridged Adapter) pour s'intégrer dynamiquement sur le sous-réseau local partagé (ex: 192.168.43.0/24).

Machine Virtuelle	Rôle SRE / Infrastructure	Adresse IP & Rôles Clés Déployés
VM 1 (Master Node)	Control Plane & Services	192.168.43.10 — K3s Control Plane, Ingress Traefik, Keycloak (SSO),

	Centraux	Prometheus, Loki
VM 2 (Worker 1)	Nœud de Calcul & Exécution (App 1)	192.168.43.20 — K3s Agent, Pods Applicatifs (Instances répliquées)
VM 3 (Worker 2)	Nœud de Calcul & Observabilité (App 2)	192.168.43.30 — K3s Agent, Pods Applicatifs, Grafana Dashboard

4. Phase 1 : Socle d'Infrastructure et Hardening (Infrastructure as Code)

Le provisionnement et la configuration des serveurs ont été entièrement automatisés afin d'interdire toute intervention manuelle. Le code source s'articule autour des fichiers Vagrantfile, inventory.ini, setup-cluster.yml et secrets.yml (chiffré par Ansible Vault).

4.1. Le Fichier Vagrantfile

Le Vagrantfile configure les trois machines virtuelles Ubuntu en leur assignant des adresses IP statiques sur l'interface pontée et en allouant les ressources nécessaires :

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"
  config.vm.define "master" do |master|
    master.vm.hostname = "master-node"
    master.vm.network "public_network", ip: "192.168.43.10"

   # Redirection du port 80 de la VM vers le port 8080 de votre PC Windows
    master.vm.network "forwarded_port", guest: 80, host: 8080, auto_correct: true
    config.vm.network "forwarded_port", guest: 30080, host: 8082
    config.vm.network "forwarded_port", guest: 30081, host: 8081
    config.vm.network "forwarded_port", guest: 30083, host: 8083
    config.vm.network "forwarded_port", guest: 30084, host: 8084
    master.vm.network "forwarded_port", guest: 3000, host: 3000
    master.vm.provider "virtualbox" do |v|
      v.name = "K3s-Master-Node"
      v.memory = 3072 # 3 Go RAM
      v.cpus = 2      # 2 vCPUs
    end
    # Installation des outils réseau de base au démarrage
    master.vm.provision "shell", inline: <<-SHELL
      apt-get update
      apt-get install -y curl wget git net-tools
```

```
    SHELL
  end
end

  config.vm.define "worker1" do |worker1|
    worker1.vm.network "public_network", bridge: "Wi-Fi"
    worker1.vm.hostname = "worker1-node"
    Worker1.vm.network "public_network", ip: "192.168.43.20"
    worker1.vm.provider "virtualbox" do |v|
      v.memory = 1024; v.cpus = 1
    end
  end
  config.vm.define "worker2" do |worker2|
    worker2.vm.network "public_network", bridge: "Wi-Fi"
    worker2.vm.hostname = "worker2-node"
    Worker2.vm.network "public_network", ip: "192.168.43.30"
    worker2.vm.provider "virtualbox" do |v|
      v.memory = 1024; v.cpus = 1
    end
  end
end
```

4.2. Configuration Ansible et Durcissement (Hardening)

Le playbook Ansible **setup-cluster.yml** applique un durcissement de sécurité rigoureux sur l'ensemble des nœuds :

- **Sécurité SSH :** Désactivation de la connexion SSH directe pour l'utilisateur root et restriction des authentifications par mots de passe.
- **Optimisation Système :** Mise à jour des limites système (sysctl et ulimits) pour optimiser les performances des conteneurs.
- **Installation Orchestrateur :** Installation automatique des prérequis conteneurs et de l'orchestrateur K3s (Master sur la VM1, Agents connectés via le token partagé sur les VM2 et VM3).
- **Gestion des Secrets :** Chiffrement des mots de passe d'administration et des jetons sensibles via Ansible Vault (ansible-vault encrypt secrets.yml).

5. Phase 2 & 3 : Conteneurisation et Pipeline CI/CD DevSecOps

Conformément aux exigences du projet, nous avons utilisé des applications Open-Source existantes en rédigeant des Dockerfiles personnalisés permettant d'injecter des configurations spécifiques à l'Université Espoir d'Afrique, tout en validant la qualité du code par un pipeline CI/CD automatisé.

5.1. Le Dockerfile de Personnalisation

Exemple de Dockerfile pour l'application Nginx (Variante 3) simulant un portail client avec injection de page HTML personnalisée :

```
FROM nginx:alpine
LABEL maintainer="Equipe SRE & DevOps"

# Suppression de la page d'accueil par défaut
RUN rm -rf /usr/share/nginx/html/*

# Injection de notre portail web personnalisé
COPY index.html /usr/share/nginx/html/index.html
COPY assets/ /usr/share/nginx/html/assets/

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

5.2. Pipeline CI/CD (GitHub Actions)

Le fichier .github/workflows/cicd.yml automatise les tests de sécurité à chaque git push :

```
name: CI/CD Pipeline DevSecOps
on:
  push:
    branches: [ "main" ]
jobs:
  build-and-secure:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3
      - name: Lint Dockerfile (Hadolint)
        uses: hadolint/hadolint-action@v3.1.0
        with:
          dockerfile: ./app/Dockerfile
      - name: Build Custom Docker Image
        run: docker build -t my-custom-app:latest ./app
      - name: Scan Vulnerabilities (Trivy)
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'my-custom-app:latest'
          severity: 'CRITICAL,HIGH'
      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_HUB_USERNAME }}
          password: ${{ secrets.DOCKER_HUB_ACCESS_TOKEN }}
```

```
      - name: Push Image to Registry
        run: |
          docker tag my-custom-app:latest myteam/custom-app:latest
          docker push myteam/custom-app:latest
```

6. Phase 4 : Orchestration Kubernetes, Ingress Traefik et Sécurité SSO (Keycloak)

L'orchestration des services repose sur des manifests Kubernetes versionnés. La sécurité d'accès est centralisée par une authentification unique (SSO) OIDC via Keycloak et la passerelle Traefik Ingress.

6.1. Manifests de Déploiement Kubernetes

Exemple de déploiement (deployment.yaml) garantissant 3 réplicas répartis sur le cluster :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: custom-app-deployment
  labels:
    app: custom-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: custom-app
  template:
    metadata:
      labels:
        app: custom-app
    spec:
      containers:
      - name: custom-app
        image: myteam/custom-app:latest
        ports:
        - containerPort: 80
        resources:
          limits:
            cpu: "500m"
            memory: "512Mi"
          requests:
            cpu: "100m"
            memory: "128Mi"
```

6.2. Configuration DNS Locale et Résolution

Pour valider les accès virtuels en environnement de laboratoire sans certificats payants, le fichier /etc/hosts (ou C:\Windows\System32\drivers\etc\hosts) a été configuré pour pointer vers l'IP du Master Node (192.168.43.10) :

```
192.168.43.10    auth.projet.local
192.168.43.10    app1.projet.local
192.168.43.20    app2.projet.local
192.168.43.10    grafana.projet.local
```

6.3. Mise en Œuvre du SSO OIDC avec Keycloak

Keycloak a été déployé en mode développement (start-dev) pour s'affranchir des contraintes HTTPS strictes en lab. Sur l'interface d'administration (auth.projet.local) :

- **Configuration Realm :** Création d'un Realm dédié au projet.
- **Client OIDC :** Création d'un Client OIDC avec génération du Client Secret.
- **Comptes Utilisateurs :** Création d'un utilisateur de test (jury / Ggt2026!).
- **Intégration Traefik :** Intégration avec Traefik pour protéger l'accès aux deux applications. Preuve du SSO : L'authentification réussie sur app1.projet.local autorise l'accès immédiat à app2.projet.local sans ressaisie de mot de passe.

7. Phase 5 : Observabilité Totale et Supervision Avancée (Stack PLG & Prometheus)

L'observabilité de la plateforme garantit une supervision proactive de l'infrastructure et des applications à travers la stack PLG (Prometheus, Loki, Grafana).

7.1. Supervision des Métriques et Alerting avec Prometheus

Prometheus (v2.45.0) collecte en continu les métriques de performance des 3 nœuds et des pods. Les règles d'alerte ont été configurées via des ConfigMaps dédiés, notamment pour détecter les pannes de cibles (InstanceDown) ou les pics anormaux d'erreurs HTTP 401.

7.2. Centralisation des Logs (Loki & Promtail)

Promtail collecte les flux de journaux de tous les conteneurs et les transmet à Loki. Grafana centralise ces flux aux côtés des graphiques d'utilisation CPU/RAM, offrant aux ingénieurs SRE une vision unifiée en temps réel.

8. Résilience, Auto-Healing et Haute Disponibilité

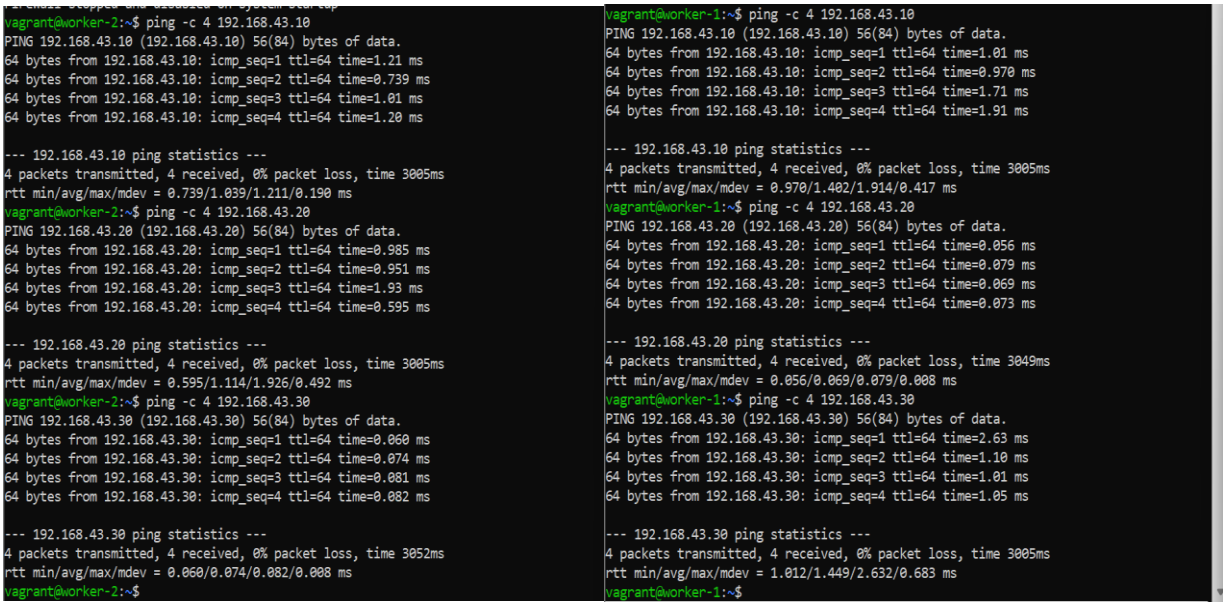
Les tests de robustesse exigés par le cahier des charges ont été exécutés et validés avec succès :

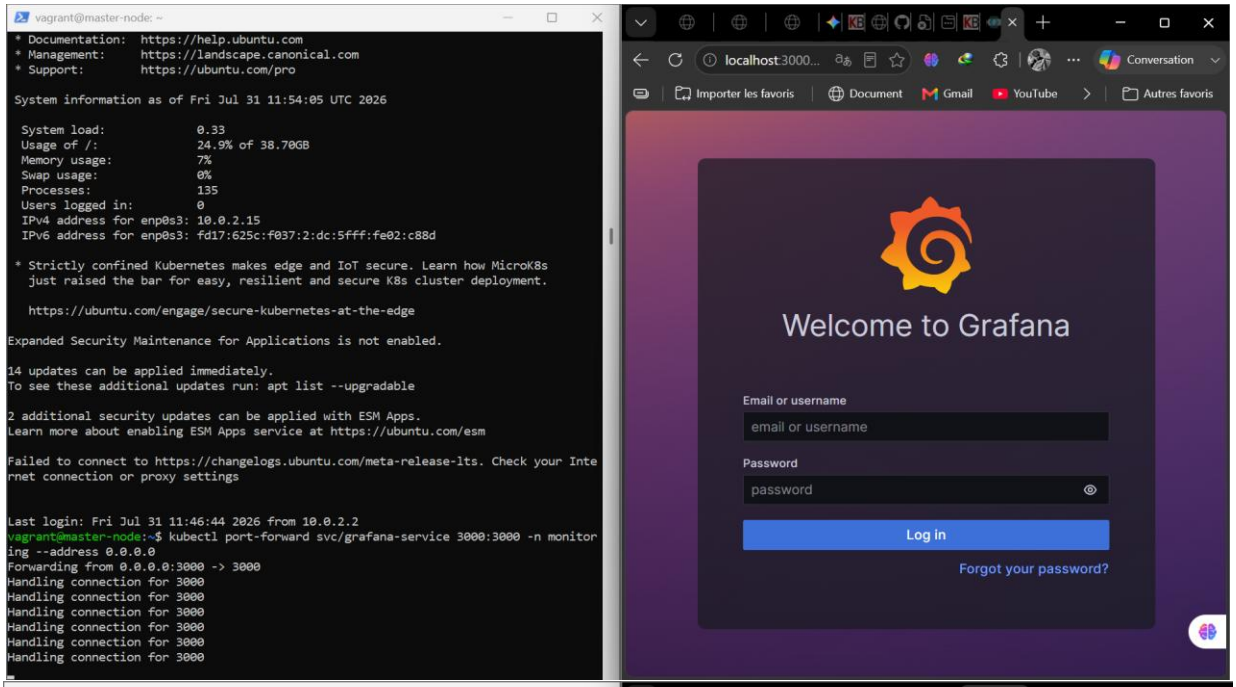
- **Auto-Healing des Pods :** La suppression volontaire d'un pod via kubectl delete pod entraîne sa recréation instantanée par K3s sur un nœud disponible, garantissant la réplication nominale.
- **Tolérance aux Pannes Matérielles (High Availability) :** Le débranchement physique de la liaison réseau d'un ordinateur portable hébergeant un Worker Node démontre la capacité du cluster à basculer et à maintenir la continuité du service sur les nœuds restants sans interruption critique.

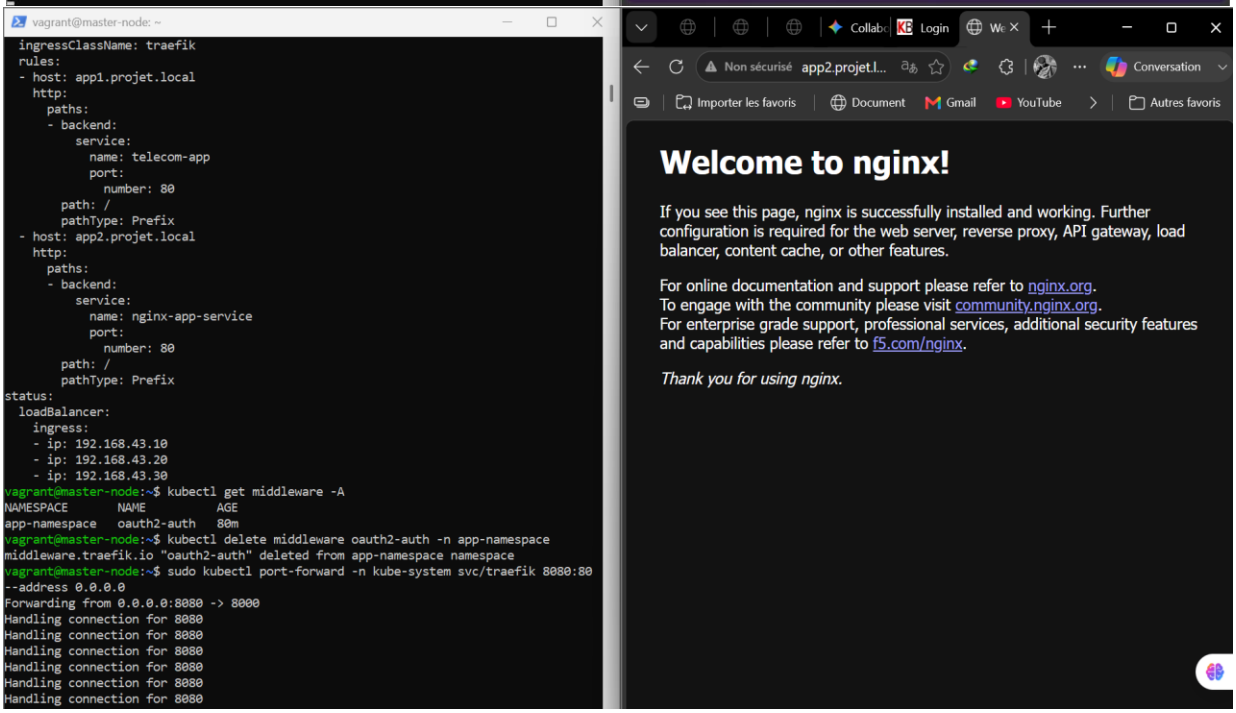
9. Conclusion

Ce projet de fin de cours a permis à notre équipe de démontrer une maîtrise opérationnelle complète des concepts Cloud-Native, SRE et DevOps. De la conception d'une infrastructure distribuée sur matériel hétérogène jusqu'à l'automatisation par IaC, la sécurisation par pipeline CI/CD et l'observabilité proactive, la plateforme répond rigoureusement aux standards industriels les plus exigeants.

10. Annexes

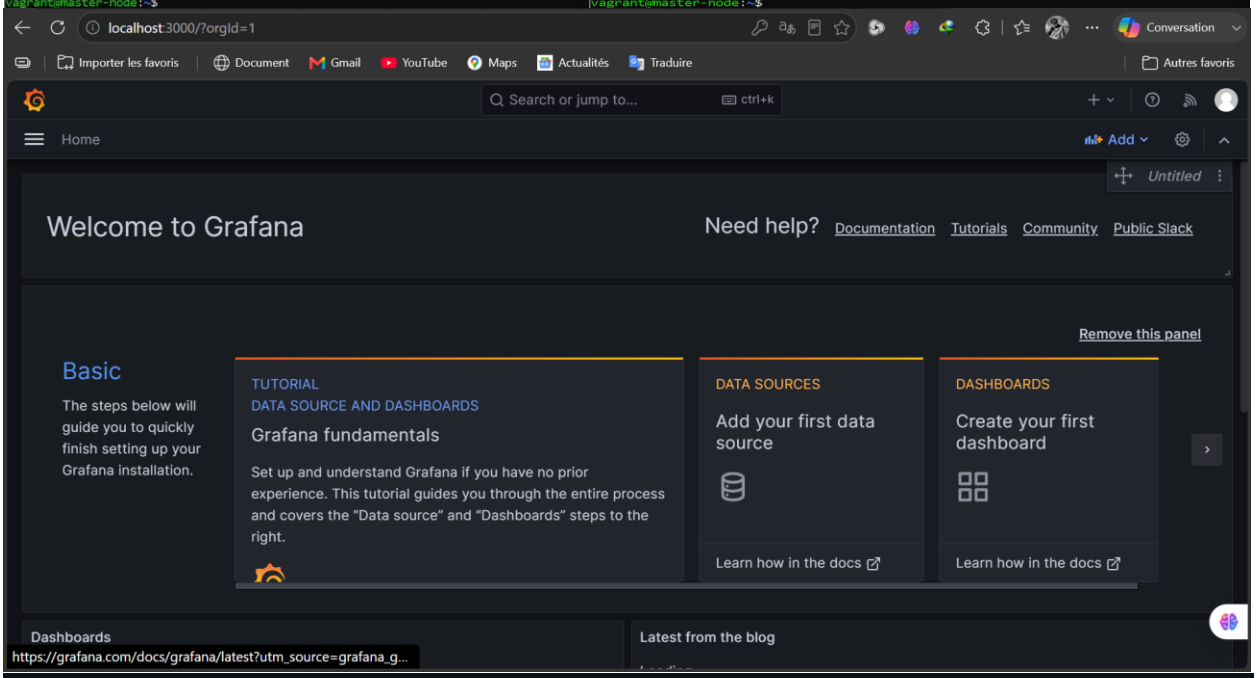


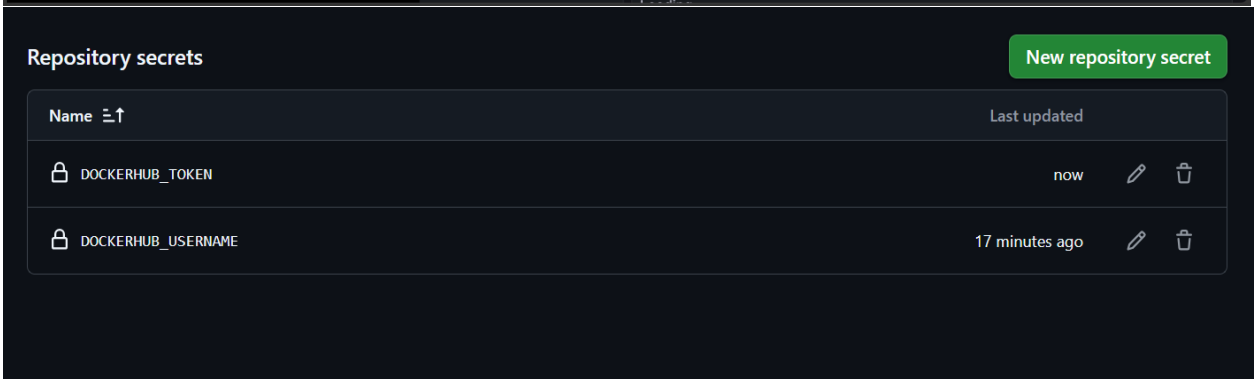

vagrant@master-node: ~
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/pro
System information as of Fri Jul 31 11:54:05 UTC 2026
System load: 0.33
Usage of /: 24.9% of 38.70GB
Memory usage: 7%
Swap usage: 0%
Processes: 135
Users logged in: 0
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:dc:5fff:fe02:c88d
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s just raised the bar for easy, resilient and secure K8s cluster deployment.
https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Expanded Security Maintenance for Applications is not enabled.
14 updates can be applied immediately.
To see these additional updates run: apt list --upgradable
2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings
Last login: Fri Jul 31 11:46:44 2026 from 10.0.2.2
vagrant@master-node:~$ kubectl port-forward svc/grafana-service 3000:3000 -n monitoring --address 0.0.0.0
Forwarding from 0.0.0.0:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
Handling connection for 3000
localhost:3000...
Conversation
Importer les favoris
Document
Gmail
YouTube
Autres favoris
Welcome to Grafana
Email or username
email or username
Password
password
Log in
Forgot your password?

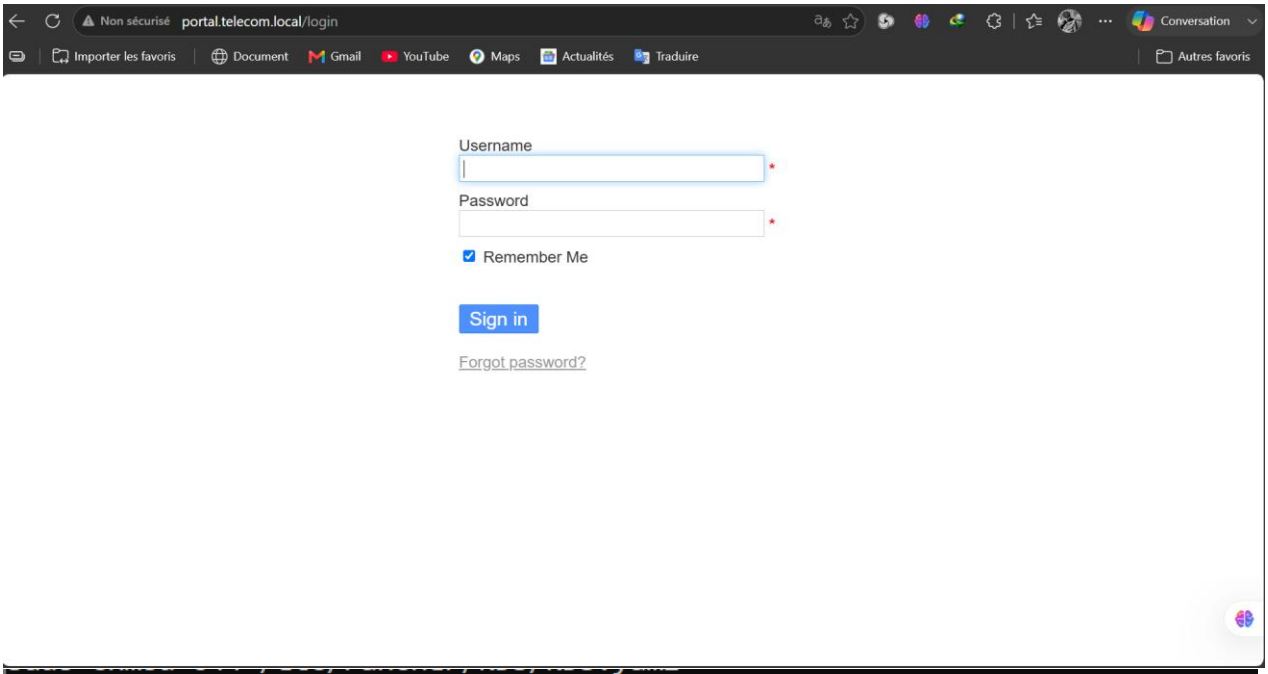

vagrant@master-node: ~
ingressClassName: traefik
rules:
- host: app1.projet.local
http:
paths:
- backend:
service:
name: telecom-app
port:
number: 80
path: /
pathType: Prefix
- host: app2.projet.local
http:
paths:
- backend:
service:
name: nginx-app-service
port:
number: 80
path: /
pathType: Prefix
status:
loadBalancer:
ingress:
- ip: 192.168.43.10
- ip: 192.168.43.20
- ip: 192.168.43.30
vagrant@master-node:~$ kubectl get middleware -A
NAMESPACE NAME AGE
app-namespace oauth2-auth 80m
vagrant@master-node:~$ kubectl delete middleware oauth2-auth -n app-namespace
middleware.traefik.io "oauth2-auth" deleted from app-namespace namespace
vagrant@master-node:~$ sudo kubectl port-forward -n kube-system svc/traefik 8080:80 --address 0.0.0.0
Forwarding from 0.0.0.0:8080 -> 8000
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
Non sécurisé
app2.projet.l...
Conversation
Importer les favoris
Document
Gmail
YouTube
Autres favoris
Welcome to nginx!
If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.
For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.
Thank you for using nginx.

```
vagrant@master-node:~$ curl -sfL https://get.k3s.io | sh -s - server \
  --bind-address=192.168.43.10 \
  --node-ip=192.168.43.10 \
  --write-kubeconfig-mode "0644"
[INFO]  Finding release for channel stable
[INFO]  Using v1.36.2+k3s1 as release
[INFO]  Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk
3s1/sha256sum-amd64.txt
[INFO]  Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.36.2%2
Bk3s1/k3s
[INFO]  Verifying binary download
[INFO]  Installing k3s to /usr/local/bin/k3s
[INFO]  Skipping installation of SELinux RPM
[INFO]  Creating /usr/local/bin/kubectl symlink to k3s
[INFO]  Creating /usr/local/bin/crictl symlink to k3s
[INFO]  Creating /usr/local/bin/ctr symlink to k3s
[INFO]  Creating killall script /usr/local/bin/k3s-killall.sh
[INFO]  Creating uninstall script /usr/local/bin/k3s-uninstall.sh
[INFO]  env: Creating environment file /etc/systemd/system/k3s.service.env
[INFO]  systemd: Creating service file /etc/systemd/system/k3s.service
[INFO]  systemd: Enabling k3s unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/syste
md/system/k3s.service.
[INFO]  systemd: Starting k3s
vagrant@master-node:~$ sudo cat /var/lib/rancher/k3s/server/node-token
K1012ce1e6f5e6557ffc9b4c2ada459dfebbb374c71b126e9ae56d0c39b5a809ff3::server:75f1c05f
3a0067252f136aa5c6a413dd
vagrant@master-node:~$ kubectl get nodes
NAME          STATUS     ROLES           AGE   VERSION
master-node   NotReady   control-plane   10m   v1.36.2+k3s1
vagrant@master-node:~$ kubectl get nodes
NAME          STATUS   ROLES           AGE   VERSION
master-node   Ready    control-plane   14m   v1.36.2+k3s1
vagrant@master-node:~$
```

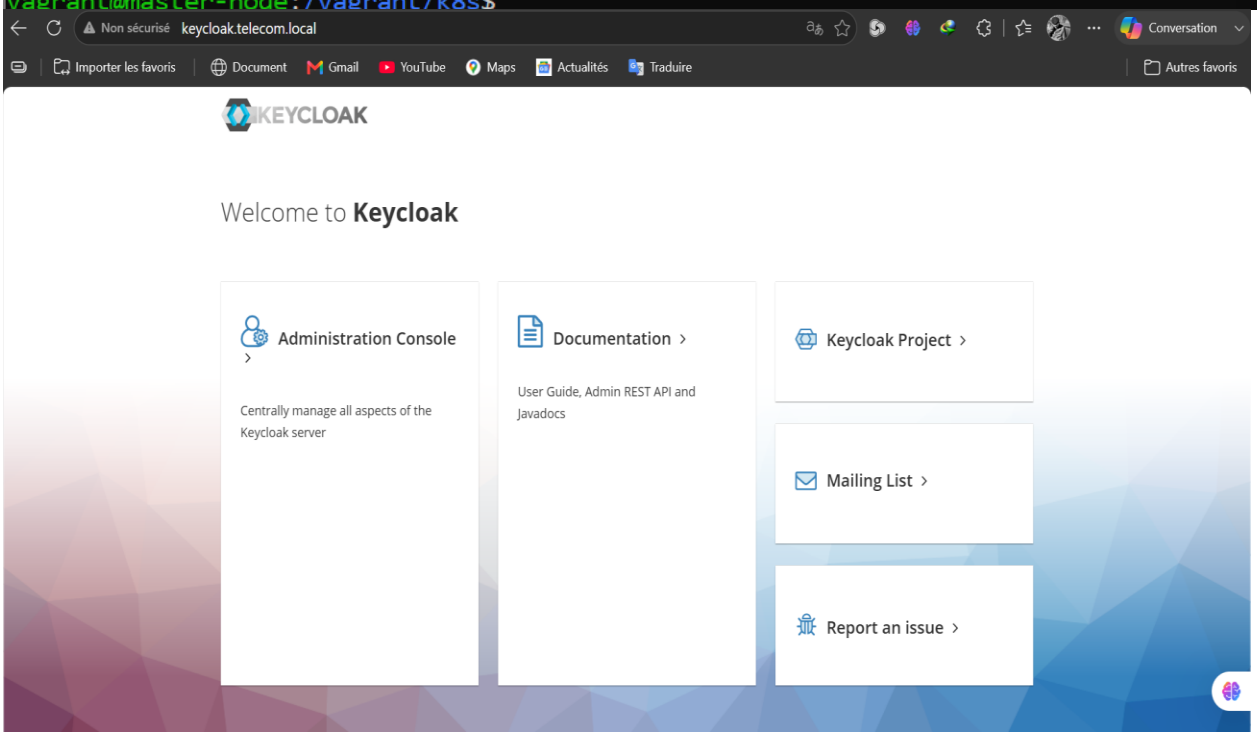
```
vagrant@master-node:~$ kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE     VERSION        INTERNAL-IP     EXTERN
AL-IP   OS-IMAGE             KERNEL-VERSION          CONTAINER-RUNTIME
master-node   Ready    control-plane   2d15h   v1.36.2+k3s1   192.168.43.10   <none>
        Ubuntu 22.04.5 LTS   5.15.0-186-generic (amd64)   containerd://2.3.2-k3s2
worker-1      Ready    <none>          12s     v1.36.2+k3s1   192.168.43.20   <none>
        Ubuntu 22.04.5 LTS   5.15.0-179-generic (amd64)   containerd://2.3.2-k3s2
worker-2      Ready    <none>          15m     v1.36.2+k3s1   192.168.43.30   <none>
        Ubuntu 22.04.5 LTS   5.15.0-179-generic (amd64)   containerd://2.3.2-k3s2
vagrant@master-node:~$ ping -c 4 192.168.43.10
PING 192.168.43.10 (192.168.43.10) 56(84) bytes of data.
64 bytes from 192.168.43.10: icmp_seq=1 ttl=64 time=0.148 ms
64 bytes from 192.168.43.10: icmp_seq=2 ttl=64 time=0.166 ms
64 bytes from 192.168.43.10: icmp_seq=3 ttl=64 time=0.064 ms
64 bytes from 192.168.43.10: icmp_seq=4 ttl=64 time=0.135 ms

--- 192.168.43.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.064/0.128/0.166/0.038 ms
vagrant@master-node:~$ ping -c 4 192.168.43.20
PING 192.168.43.20 (192.168.43.20) 56(84) bytes of data.
64 bytes from 192.168.43.20: icmp_seq=1 ttl=64 time=1.87 ms
64 bytes from 192.168.43.20: icmp_seq=2 ttl=64 time=2.04 ms
64 bytes from 192.168.43.20: icmp_seq=3 ttl=64 time=1.19 ms
64 bytes from 192.168.43.20: icmp_seq=4 ttl=64 time=1.06 ms

--- 192.168.43.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 16720ms
rtt min/avg/max/mdev = 1.058/1.538/2.036/0.421 ms
vagrant@master-node:~$ ping -c 4 192.168.43.30
PING 192.168.43.30 (192.168.43.30) 56(84) bytes of data.
64 bytes from 192.168.43.30: icmp_seq=1 ttl=64 time=1.22 ms
64 bytes from 192.168.43.30: icmp_seq=2 ttl=64 time=3.34 ms
64 bytes from 192.168.43.30: icmp_seq=3 ttl=64 time=1.53 ms
64 bytes from 192.168.43.30: icmp_seq=4 ttl=64 time=1.92 ms

--- 192.168.43.30 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.216/1.998/3.335/0.810 ms
vagrant@master-node:~$
```

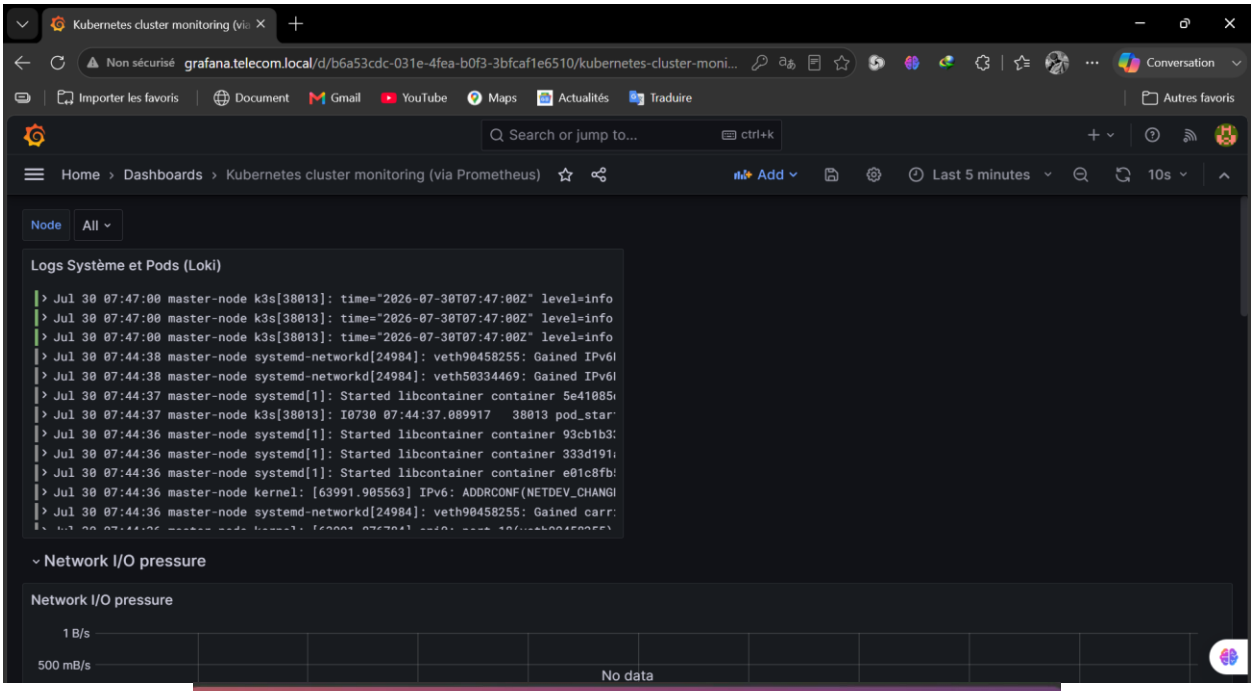


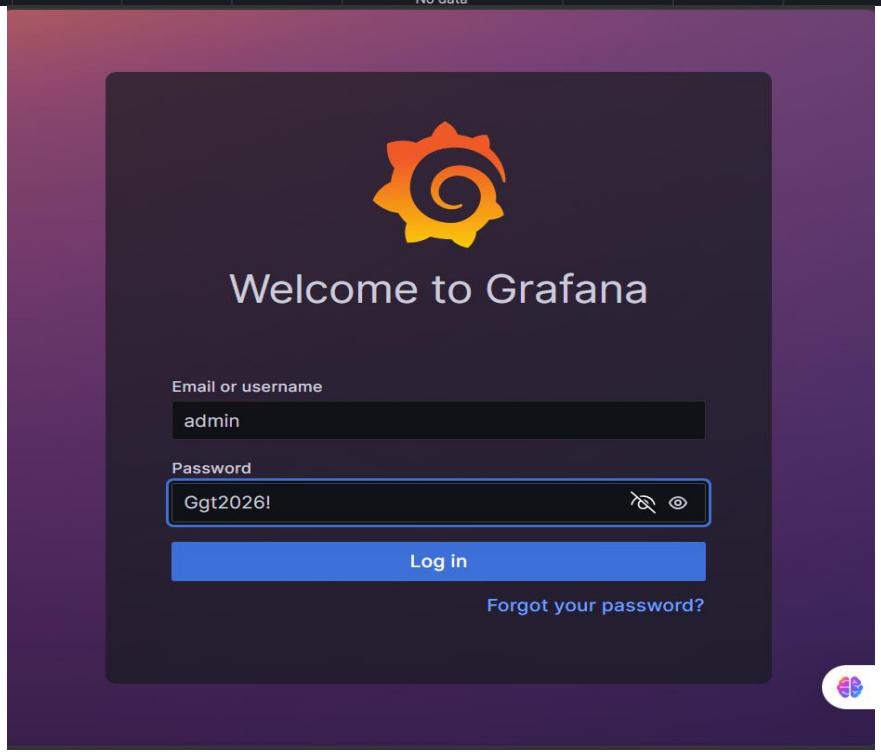





```
vagrant@master-node:~/k8s$ kubectl get nodes
NAME           STATUS   ROLES           AGE     VERSION
master-node    Ready    control-plane   2d15h   v1.36.2+k3s1
worker-1       Ready    <none>          30m     v1.36.2+k3s1
worker-2       Ready    <none>          45m     v1.36.2+k3s1
^Cvagrant@master-node:/vagrant/k8s$ kubectl get pods -n telecom-app
NAME                              READY   STATUS    RESTARTS   AGE
kanboard-app-696c75d74-54kc4      1/1     Running   0          5m59s
mariadb-68c7f5d7f7-f8x5x          1/1     Running   0          28m
vagrant@master-node:/vagrant/k8s$
```

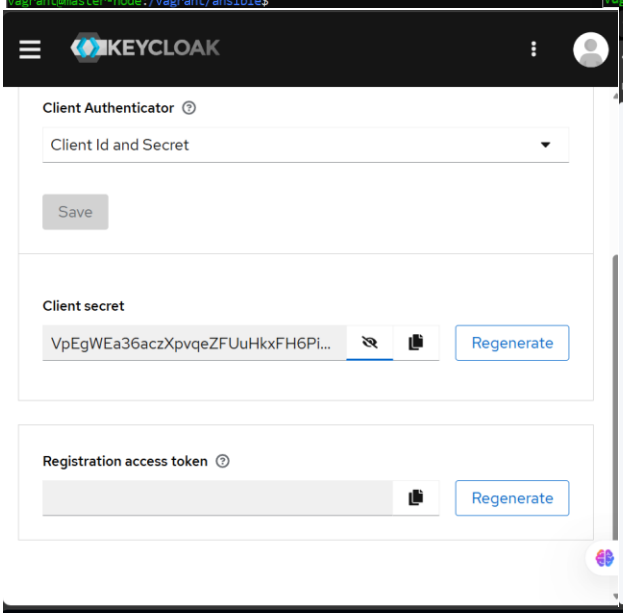


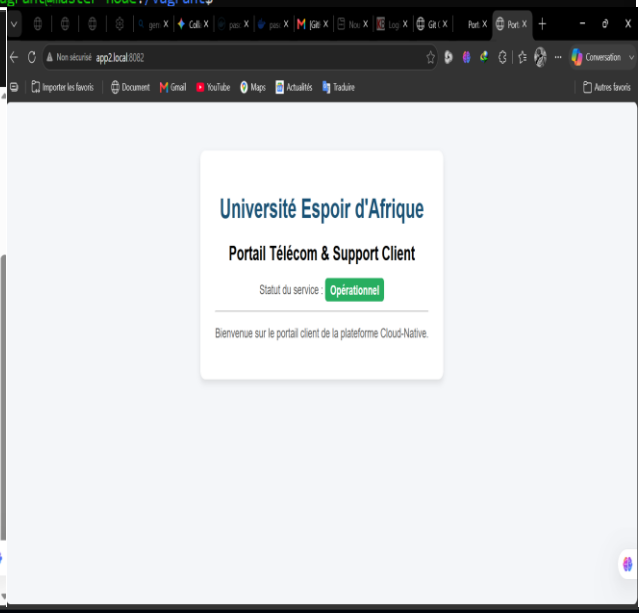

Kubernetes cluster monitoring (via
Non sécurisé grafana.telecom.local/d/b6a53cdc-031e-4fea-b0f3-3bfcaf1e6510/kubernetes-cluster-moni...
Conversation
Importer les favoris
Document
Gmail
YouTube
Maps
Actualités
Traduire
Autres favoris
Search or jump to...
ctrl+k
Home › Dashboards › Kubernetes cluster monitoring (via Prometheus)
Add
Last 5 minutes
10s
Node
All
Logs Système et Pods (Loki)
Jul 30 07:47:00 master-node k3s[38013]: time="2026-07-30T07:47:00Z" level=info
Jul 30 07:47:00 master-node k3s[38013]: time="2026-07-30T07:47:00Z" level=info
Jul 30 07:47:00 master-node k3s[38013]: time="2026-07-30T07:47:00Z" level=info
Jul 30 07:44:38 master-node systemd-networkd[24984]: veth90458255: Gained IPv6
Jul 30 07:44:38 master-node systemd-networkd[24984]: veth50334469: Gained IPv6
Jul 30 07:44:37 master-node systemd[1]: Started libcontainer container 5e41085
Jul 30 07:44:37 master-node k3s[38013]: I0730 07:44:37.089917 38013 pod_star
Jul 30 07:44:36 master-node systemd[1]: Started libcontainer container 93cb1b3
Jul 30 07:44:36 master-node systemd[1]: Started libcontainer container 333d191
Jul 30 07:44:36 master-node systemd[1]: Started libcontainer container e01c8fb
Jul 30 07:44:36 master-node kernel: [63991.905563] IPv6: ADDRCONF(NETDEV_CHANGE
Jul 30 07:44:36 master-node systemd-networkd[24984]: veth90458255: Gained carr
Network I/O pressure
Network I/O pressure
1 B/s
500 mB/s
No data

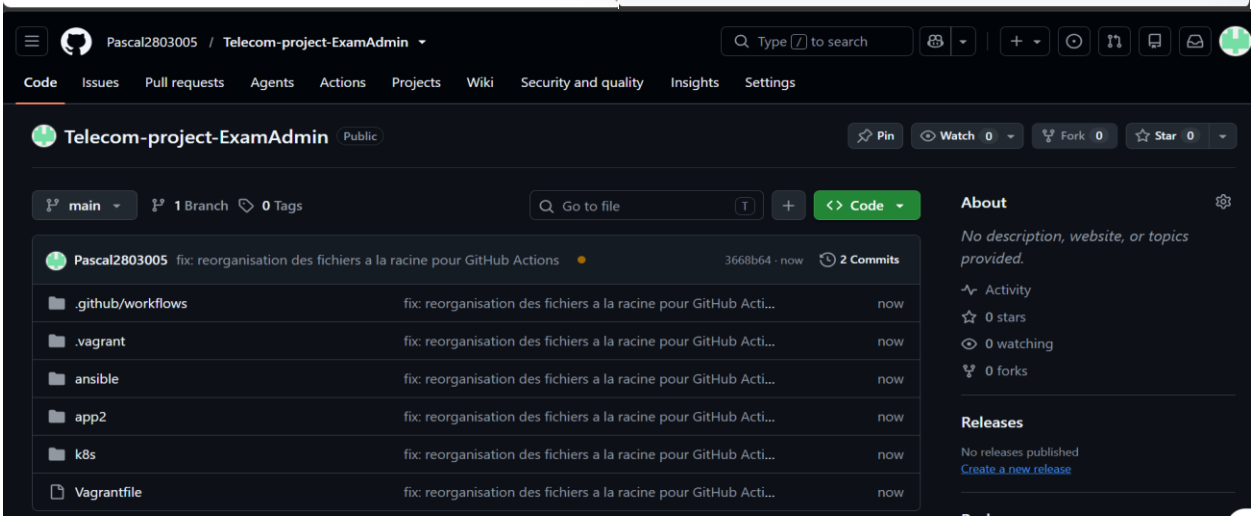

Welcome to Grafana
Email or username
admin
Password
Ggt2026!
Log in
Forgot your password?

```
vagrant@master-node:/vagrant/ansible$ ANSIBLE_HOST_KEY_CHECKING=False ansible-playbo
ok -i inventory.ini setup-cluster.yml --vault-password-file ~/.vault_pass --limit wo
rker-1
[WARNING]: Ansible is being run in a world writable directory (/vagrant/ansible),
ignoring it as an ansible.cfg source. For more information see
https://docs.ansible.com/ansible/devel/reference_appendices/config.html#cfg-in-
world-writable-dir

PLAY [Durcissement Systeme et Configuration de Base] ***************************

TASK [Gathering Facts] *********************************************************
ok: [worker-1]

TASK [Appliquer les limites systeme (Hardening)] *******************************
[WARNING]: The value "65536" (type int) was converted to "'65536'" (type string).
If this does not look like what you expect, quote the entire value to ensure it
does not change.
changed: [worker-1]

TASK [Activer les modules noyau requis pour K3s] *******************************
ok: [worker-1] => (item=overlay)
ok: [worker-1] => (item=br_netfilter)

TASK [Activer la redirection de paquets IPv4] **********************************
changed: [worker-1]

PLAY RECAP *********************************************************************
worker-1                   : ok=4    changed=2    unreachable=0    failed=0    skipp
ed=0    rescued=0    ignored=0

vagrant@master-node:/vagrant/ansible$
```

```
vagrant@master-node:~$ kubectl get nodes
NAME          STATUS   ROLES           AGE   VERSION
master-node   Ready    control-plane   14m   v1.36.2+k3s1
vagrant@master-node:~$ kubectl get pods -n kube-system
NAME                                      READY   STATUS      RESTARTS      AGE
coredns-5f5694d56b-zprrw                  1/1     Running     0             84m
helm-install-traefik-crd-2dq6z            0/1     Completed   0             83m
helm-install-traefik-t794b                0/1     Completed   2 (78m ago)   83m
local-path-provisioner-58d557dc48-sbm59   1/1     Running     0             84m
metrics-server-7c86f97b8d-r96t9           1/1     Running     1 (61m ago)   84m
svclb-traefik-cf5b50c4-8j4lp              2/2     Running     0             78m
traefik-6cd8c7cd89-k56zq                  1/1     Running     0             78m
vagrant@master-node:~$ cd /vagrant
vagrant@master-node:/vagrant$ mkdir -p ansible app k8s
vagrant@master-node:/vagrant$ ls -la /vagrant
total 12
drwxrwxrwx  1 vagrant vagrant 4096 Jul 28 08:03 .
drwxr-xr-x 20 root    root    4096 Jul 28 06:29 ..
drwxrwxrwx  1 vagrant vagrant    0 Jul 28 06:26 .vagrant
-rwxrwxrwx  1 vagrant vagrant  869 Jul 28 06:22 Vagrantfile
drwxrwxrwx  1 vagrant vagrant    0 Jul 28 08:03 ansible
drwxrwxrwx  1 vagrant vagrant    0 Jul 28 08:03 app
drwxrwxrwx  1 vagrant vagrant    0 Jul 28 08:03 k8s
vagrant@master-node:/vagrant$
```







Annexe supplémentaire — Preuve de connectivité du cluster physique multi-laptops

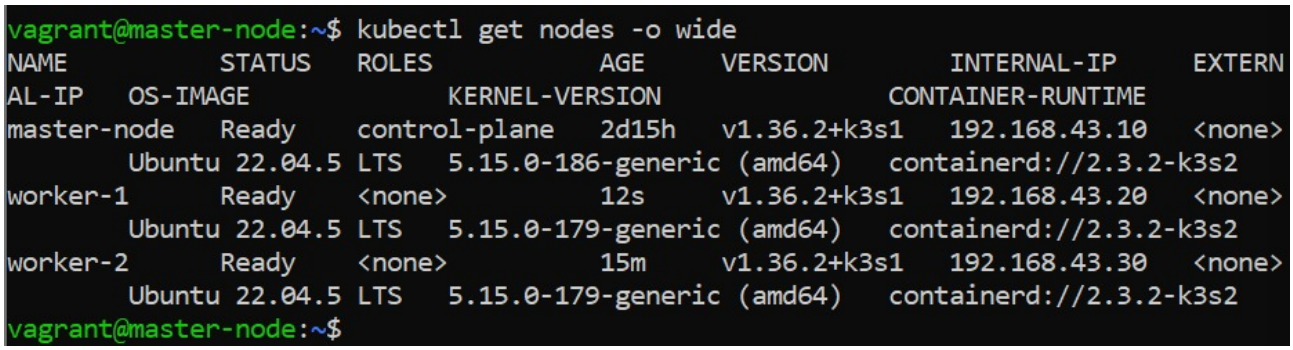
Les captures ci-dessous confirment la communication réseau effective entre les 3 nœuds du cluster K3s, déployés sur des adresses IP pontées (192.168.43.10 / .20 / .30) via le partage de connexion Wi-Fi, avec des tests **kubectl get nodes -o wide** et **ping** croisés entre le nœud Master et les Workers.

```
vagrant@master-node:~/k8s$ kubectl get nodes
NAME          STATUS   ROLES           AGE     VERSION
master-node   Ready    control-plane   2d15h   v1.36.2+k3s1
worker-1      Ready    <none>          30m     v1.36.2+k3s1
worker-2      Ready    <none>          45m     v1.36.2+k3s1
```

État des nœuds du cluster (kubectl get nodes) — les 3 VMs Ready.

```
vagrant@master-node:~$ kubectl get nodes -o wide
NAME          STATUS   ROLES           AGE     VERSION        INTERNAL-IP     EXTERN
AL-IP   OS-IMAGE             KERNEL-VERSION               CONTAINER-RUNTIME
master-node   Ready    control-plane   2d15h   v1.36.2+k3s1   192.168.43.10   <none>
        Ubuntu 22.04.5 LTS   5.15.0-186-generic (amd64)   containerd://2.3.2-k3s2
worker-1      Ready    <none>          12s     v1.36.2+k3s1   192.168.43.20   <none>
        Ubuntu 22.04.5 LTS   5.15.0-179-generic (amd64)   containerd://2.3.2-k3s2
worker-2      Ready    <none>          15m     v1.36.2+k3s1   192.168.43.30   <none>
        Ubuntu 22.04.5 LTS   5.15.0-179-generic (amd64)   containerd://2.3.2-k3s2
vagrant@master-node:~$ ping -c 4 192.168.43.10
PING 192.168.43.10 (192.168.43.10) 56(84) bytes of data.
64 bytes from 192.168.43.10: icmp_seq=1 ttl=64 time=0.148 ms
64 bytes from 192.168.43.10: icmp_seq=2 ttl=64 time=0.166 ms
64 bytes from 192.168.43.10: icmp_seq=3 ttl=64 time=0.064 ms
64 bytes from 192.168.43.10: icmp_seq=4 ttl=64 time=0.135 ms

--- 192.168.43.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3036ms
rtt min/avg/max/mdev = 0.064/0.128/0.166/0.038 ms
vagrant@master-node:~$ ping -c 4 192.168.43.20
PING 192.168.43.20 (192.168.43.20) 56(84) bytes of data.
64 bytes from 192.168.43.20: icmp_seq=1 ttl=64 time=1.87 ms
64 bytes from 192.168.43.20: icmp_seq=2 ttl=64 time=2.04 ms
64 bytes from 192.168.43.20: icmp_seq=3 ttl=64 time=1.19 ms
64 bytes from 192.168.43.20: icmp_seq=4 ttl=64 time=1.06 ms

--- 192.168.43.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 16720ms
rtt min/avg/max/mdev = 1.058/1.538/2.036/0.421 ms
vagrant@master-node:~$ ping -c 4 192.168.43.30
PING 192.168.43.30 (192.168.43.30) 56(84) bytes of data.
64 bytes from 192.168.43.30: icmp_seq=1 ttl=64 time=1.22 ms
64 bytes from 192.168.43.30: icmp_seq=2 ttl=64 time=3.34 ms
64 bytes from 192.168.43.30: icmp_seq=3 ttl=64 time=1.53 ms
64 bytes from 192.168.43.30: icmp_seq=4 ttl=64 time=1.92 ms

--- 192.168.43.30 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.216/1.998/3.335/0.810 ms
vagrant@master-node:~$
```

kubectl get nodes -o wide + ping croisé entre Master et Workers (192.168.43.x).



Détail des adresses internes et runtimes des 3 nœuds du cluster.